AI / ML Guide

For whoever owns the tutor engine - on this team that is the same person who owns the backend. The engine lives as plain Python functions in `backend/app/services/` and prompt files in `prompts/`, kept separate from the HTTP routes so the two can be built and tested independently.

You own: retrieval, the alignment score, the refusal rule, the graded-work guardrail, the misconception matcher, prompts, and the provider layer.

Setup

```
git clone <repo-url>
cd ai_tutor
cp .env.example .env      # paste the keys from the team channel
./init.sh
```

Check the providers are wired:

```
.venv/Scripts/python.exe -m uvicorn app.main:app --port 8000 --app-dir backend
curl http://localhost:8000/meta/provider-status
```

Should print `{"active":"glm","fallbacks_available":["gemini","groq"],"cache_enabled":true}`.

The interface with backend

You do not write HTTP routes. You write functions. Agree the signature in one message, then work independently.

```
# backend/app/services/tutor.py

def explain(db, *, question: str, topic_id: int | None,
            language: str = "en") -> TutorResult:
    ...

def diagnose_misconception(db, *, attempt_id: int) -> Diagnosis | None:
    ...
```

`TutorResult` is a plain dataclass that mirrors the `TutorResponse` shape in `docs/api-contract.md`. Backend converts it to JSON. While you are still building, backend wraps a canned version of your function so their route works.

Three things that decide the score

These are not extras. They are the rubric. Build them first, by hour 8.

1. Alignment score

Retrieval similarity plus one cheap entailment check, expressed as a percentage on every explanation.

```
def alignment(chunks, answer_text) -> EvidenceReport:
    top = max(c.similarity for c in chunks) if chunks else 0.0
    # one LLM call: does the answer follow from these chunks? 0.0 - 1.0
    entail = llm_entailment(chunks, answer_text)
    score = 0.6 * top + 0.4 * entail
```

```
    return EvidenceReport(
        alignment_score=score,
        alignment_percent=round(score * 100),
        top_similarity=top,
        threshold=THRESHOLD,
        sufficient=score >= THRESHOLD,
        reason=None if score >= THRESHOLD else "no_matching_material",
    )
```

Calibrating the threshold - do this, do not guess

Embedding similarity has a high floor. Measured on bge-small-en-v1.5:

covered "why no net force at constant speed?" 0.78
covered "how do I split a vector?" 0.73
OFF-TOPIC "explain Lagrangian mechanics" 0.72 <-- nearly identical
OFF-TOPIC "what is photosynthesis?" 0.54
OFF-TOPIC "who won the 2018 world cup?" 0.40

Two things follow. A threshold of 0.35 would never refuse anything - the feature would silently never fire. And retrieval similarity **alone cannot separate** a near-domain off-topic question from a covered one; that is what the entailment call is for. Do not simplify it away.

Also apply the BGE query prefix to queries only:

QUERY_PREFIX = "Represent this sentence for searching relevant passages: "

That widens the covered/off-topic margin from +0.012 to +0.050 - four times the separation for one concatenation. Never prefix stored documents.

Run `backend/scripts/calibrate_threshold.py` after ingesting the real corpus and use the number it recommends.

Remember `cosine_distance` returns **distance**, so similarity is `1 - distance`. Getting this backwards inverts the score silently. Test it on a question you know is covered and one you know is not.

2. Refuse when there is no evidence

If `sufficient` is false, do not answer. Return the refusal outcome **and write an `UncertaintyFlag` row**.

```
  if not report.sufficient:
      flag = UncertaintyFlag(question=question, alignment_score=report.alignment_score,
                             reason=report.reason, topic_id=topic_id)
      db.add(flag); db.flush()
      return TutorResult(outcome="insufficient_evidence", body=REFUSAL_TEXT,
                         citations=[], evidence=report, uncertainty_flag_id=flag.id)
```

That single write also populates the teacher dashboard's Uncertainty Flags panel. One feature, two dashboards, no extra wiring.

Do **not** store a `user_id` on the flag. Teacher views are anonymous by construction.

3. Refuse graded work

If the question matches a chunk from a material with `kind="assignment"`, refuse and give hints instead.

```
  def is_graded_work(db, question) -> Material | None:
      hits = search(db, question, kinds=["assignment"], limit=1)
      return hits[0].material if hits and hits[0].similarity > 0.80 else None
```

Threshold high - you want near-verbatim matches, not "this is about the same topic". A student asking a conceptual question on a topic that also appears in an assignment must still get a real answer.

Retrieval

Brute-force cosine over pgvector. **No index** - the corpus is small and an index is a failure mode.

```
def search(db, query: str, *, limit=5, kinds=None):
    vec = embed(query)
    stmt = (select(Chunk, Chunk.embedding.cosine_distance(vec).label("dist"))
            .order_by("dist").limit(limit))
    if kinds:
        stmt = stmt.join(Material).where(Material.kind.in_(kinds))
    else:
        stmt = stmt.join(Material).where(Material.kind != "assignment")
    return [Hit(chunk=c, similarity=1 - d) for c, d in db.execute(stmt).all()]
```

Note the default excludes assignments - they are matchable but never quotable.

Embeddings are `bge-small-en-v1.5`, 384 dimensions, run through **fastembed** (ONNX, ~150MB, no PyTorch). Model files download on first use.

Use the query prefix on questions and never on stored chunks:

```
QUERY_PREFIX = "Represent this sentence for searching relevant passages: "

def embed_query(q):     return _embed(QUERY_PREFIX + q)
def embed_document(d):  return _embed(d)
```

Vectors come out unit-normalised, so cosine similarity equals the dot product.

Re-embedding rewrites the shared database for everyone - announce it first.

The provider layer

Every model call and every translation goes through `backend/app/providers/`. No direct HTTP to GLM or Sarvam from a service.

```
class Provider(Protocol):
    def complete(self, prompt: str, *, system: str = "",
                 json_schema: dict | None = None) -> str: ...
```

Implementations: `glm.py`, `gemini.py`, `groq.py`, `mock.py`. Selected by `PROVIDER` in `.env`, with automatic fallback on failure.

Cache everything. This is the single most valuable thing you will build.

```
key = sha256(f"{model}\n{system}\n{prompt}".encode()).hexdigest()
path = LLM_CACHE_DIR / f"{key}.json"
if path.exists():
    return json.loads(path.read_text())["text"]
```

Why it matters: the demo replays instantly and identically, and it survives the venue wifi dying. With `PROVIDER=mock` plus a warm cache, the entire golden path runs offline. Warm the cache by rehearsing - every rehearsal makes the real demo faster and safer.

Ask for structured output, not prose. Every "smart" function returns an object with a confidence attached.

Prompts

Plain markdown in `prompts/`. Loaded by name, so you can iterate without touching Python.

```
prompts/
  tutor_explain.md
  gap_diagnose.md
  misconception_diagnose.md
  practice_generate.md
  evidence_check.md
  reteach_suggest.md
```

Every prompt that produces student-facing text must include these rules:

```
Use ONLY the provided source material. Do not use outside knowledge.
If the sources do not contain the answer, say so - do not guess.
Cite the page number for each claim.
Never provide a final answer to a graded assignment question.
Return JSON matching the given schema. No prose outside the JSON.
```

Misconception diagnosis - the demo's closing beat

When a student answers wrong, name the **specific** wrong mental model behind that answer, then ask them to confirm.

Two-stage, cheapest first:

1. **Pattern match.** Seeded `Misconception` rows have `wrong_answer_pattern`. If the student's answer matches for that `problem_type`, you have the diagnosis with no LLM call. Fast and reliable.
2. **LLM fallback.** No pattern matched - ask the model to pick from the known misconceptions for that `problem_type`. Give it the list. Do not let it invent new ones during the demo.

```
def diagnose(db, attempt):
    for m in known_for(db, attempt.item.problem_type):
        if m.wrong_answer_pattern and matches(attempt.answer, m.wrong_answer_pattern):
            return record(db, attempt, m)
    return llm_pick(db, attempt, known_for(db, attempt.item.problem_type))
```

Then show a confirm/deny. **Only confirmed diagnoses feed the teacher heatmap.**

This lands well only if the misconception is *specific*. "Struggles with forces" is worthless. "Treats constant velocity as implying a net force" is a real diagnosis a physics teacher recognises. Pick a subject with well-documented misconceptions - mechanics and intro calculus both qualify - and seed 8 to 10 good ones with matching practice items.

Multilingual

Translate in, retrieve in English, answer in English, translate out. The vector space stays English-only, which is why we do not need multilingual embeddings.

```
q_en = translate(question, to="en") if language != "en" else question
result = explain_en(q_en)
result.body = translate(result.body, to=language) if language != "en" else result.body
```

Citations always reference the English source book and page, and the alignment score is computed on the English text. That means the score does not drift between languages - which is the correct behaviour and worth saying out loud in the pitch.

What to build in what order

1. Provider interface, cache, fallback, mock. Everything depends on it, and it is your insurance policy.
2. Retrieval with citations.
3. Alignment score.
4. Refusal plus uncertainty flag.
5. Graded-work guardrail.
6. Misconception diagnosis.
7. Practice generation.
8. Translation.
9. Reteach suggestion.

Items 3, 4, and 5 are the rubric. If you run out of time, they are the last things to cut, not the first.

Testing without burning tokens

Set `PROVIDER=mock` and write tests against fixed inputs. The suite must run with no network - that is a rule for the whole repo, so a red test means someone broke something rather than the wifi being down.